

COURTFLOW-GS: Online Multi-View Dynamic Gaussian Reconstruction with Fused MLS Tracking

Sharan Shankar

September 27, 2026

Abstract

COURTFLOW-GS reconstructs a frame-0 canonical Gaussian scene from a calibrated basketball-camera ring, then tracks dynamic motion with group-aware control points, fused rigid moving-least-squares (MLS), mask/centroid constraints, focused refinement, and periodic keyframes. The final 300-frame diagnostic produces novel-view videos and baked Gaussian checkpoints. All timing below is measured on an RTX 3080 12 GB; RTX PRO 6000 fields are intentionally reserved for replication.

1 Problem and data

The data contains 36 synchronized 4K ring cameras with fixed known calibration. Views $0, 3, \dots, 33$ train the model; the other 24 are held out. Camera poses are never optimized. The raw-capture adapter accepts `cameras/view_XXX.mp4` and `calibration/cameras.txt`, `images.txt`, producing half-resolution pinhole inputs.

2 Methodology

This section specifies the reported configuration so that the method can be recreated without reading the implementation. Let $P_c = K_c[R_c | t_c]$ be a fixed undistorted pinhole camera, with world-to-camera convention $x_c = R_c x + t_c$. Camera poses, intrinsics, and photometric gains are never optimized. RGB is divided by 255. The 12 training cameras are every third camera of the 36-camera ring; all other cameras are held out. Every input is undistorted before segmentation, matching, training, or evaluation.

2.1 Canonical frame-0 reconstruction

Why masks and 3D identities are necessary. Photometric reconstruction alone is too ambiguous for the moving players: each player occupies a small portion of a frame, shares colour and texture with teammates, and is frequently overlapped by another player or the basket structure. A foreground mask supplies three otherwise missing signals: it removes background rays from player correspondence search, constrains Gaussian alpha to the observed silhouette, and, once given a persistent identity, tells the optimizer which Gaussian/control-point group should explain each pixel. This prevents a global reconstruction from improving RGB error by assigning one player’s appearance or motion to another player or to the background.

Independent per-camera detector/SAM2 tracks are not sufficient because their instance numbers have no cross-view meaning. We first tried centroid-ray/epipolar matching with colour histograms; with 12 views about 30° apart, it fragmented 68 masks into 60 apparent identities. We also found that full-frame MAST3R is too low-resolution for the players, producing ray-like geometry. CourtFlow-GS therefore obtains identity *in 3D first*: Faster R-CNN boxes (score ≥ 0.5) are refined with SAM2, their union masks carve a court-aligned 5 cm occupancy volume, and its connected components define candidate physical players. Components are then

projected back to each camera and Hungarian-matched to the local SAM2 masks; missed detections are re-prompted from the projection. This turns noisy local masks into a single, stable player ID across all views.

The reconstruction uses a 2.4 m-high, 1 m-padded court volume; a voxel needs support in at least $\max(4, n_{\text{visible}} - 2)$ views. Implausible components are rejected by size, height, and floor contact, and wide footprints are split in the floor plane. A re-prompt is accepted only when its HSV appearance agrees with the same 3D component in other views. If it does not, that camera is left unlabelled rather than treating the player as background. Depth-ordered overlap resolution produces one mutually exclusive label image L_c per camera, which is what makes the later per-instance mask loss and separate MLS groups meaningful.

Occlusion-aware visual hull. The conventional visual-hull rule is an intersection: remove a voxel whenever any calibrated view does not see it in the silhouette. It is effective for clean turntable captures but is too strict for a game: the basket stanchion, padding, and nearer players hide genuine heads, limbs, and feet. We observed exactly that failure with a strict hull. The opposite workaround—allowing arbitrary missing views—was worse: one allowed miss inflated player hulls by roughly $2.5\times$. The chosen rule therefore keeps the precision of a strict hull while exempting a disagreement only when a nearer reconstructed surface gives a physical explanation for it.

For identity k , make a 1 cm grid in the coarse-component bounds plus two coarse-voxel margins. Camera c gives a candidate voxel x a negative vote only if x is in its frustum, c has an accepted label for k , $L_c(\pi_c(x)) \neq k$, and no nearer reconstructed surface explains the absence. The absence is explained if either (i) the pixel belongs to another labelled identity j with $d_{c,j} < d_{c,k}$, or (ii) a static depth map obeys $D_c(\pi_c(x)) < z_c(x) - 0.30$ m. Build D_c from full-frame MAST3R points labelled background and colour-consistent in at least two extra views, keeping nearest depth after a 4-pixel square splat; exclude points within 0.5 m of the floor.

Retain x iff it is observed inside k ’s label in at least two cameras and has zero unexplained negative votes; then remove voxels below the floor. Therefore a missing silhouette pixel is not forgiven merely because occlusion is possible: it is forgiven only when a measured nearer dynamic or static depth demonstrates an occluder. An unlabelled camera casts no vote. This is the precise meaning of “occlusion-aware” in this method.

Masked crop geometry and Gaussian seeds. Standard full-frame feature matching works well for the court and background, but not for a player that is only about 6×25 pixels at MAST3R’s 512-pixel full-frame resolution. Our full-frame player matches produced a radial “star” of unreliable rays. The identity masks let the method allocate descriptor resolution where it matters: CourtFlow-GS crops a padded square around the same 3D identity in each adjacent/skip-one camera pair and resizes it to 512×512 . Reciprocal MAST3R matching is seeded only inside the source mask. After mapping matches back to the original cameras, retain only confident, positive-depth, low-reprojection-error, sufficiently-wide-baseline points inside the person’s dilated visual hull that agree in at least one extra labelled view. This rejects the floor/background rays that a mask majority vote admitted in earlier experiments.

The retained points are accurate surface seeds; surface hull voxels only fill gaps farther than two hull voxels from a seed, where texture was inadequate for matching. We colour points from label-consistent, self-z-buffer-visible views, remove outliers, fuse people at 5 mm and background at 10 cm, then initialize normal-oriented, anisotropic degree-1-SH Gaussians. Persons train for 7000 iterations (four cameras/step) and background for 10000 (two cameras/step), using

$$\mathcal{L}_{\text{image}} = \|I - \hat{I}\|_1 + 0.2 \text{D-SSIM} + 0.1 \mathcal{L}_{\text{alpha}} + 0.05 \mathcal{L}_{\text{depth}}.$$

Depth weight decays to zero halfway through training. Densification starts at iteration 500 and runs every 100 iterations until halfway; prune opacity below 0.01 and splats outside a 3-pixel-dilated mask in two views.

2.2 Group-aware control-point MLS

Why group-aware deformation. A conventional global deformation graph, including the all-controls weighting used by methods such as QuickCapture, is appropriate for one connected deforming object. It is unsafe for a crowded multi-person scene: nearby controls on different players can explain the same image error, so a player may pull a teammate or a background splat. CourtFlow-GS treats each identity as a separate deformation domain, preserving the useful local smoothness of MLS without allowing cross-player coupling. Compact eight-control support also keeps deformation memory and work linear in the number of Gaussians.

Every dynamic Gaussian belongs to exactly one person (the ball, when enabled, is a separate group); background is group 0. Farthest-point sample 1024 controls per person, 16 for the ball, and 1000 for background. Each Gaussian is bound once to its $K = 8$ nearest controls from the *same* group. Its fixed radial-basis weights are

$$w_{ij} = \frac{\exp(-\|x_i - p_j\|^2 / (2\sigma_j^2))}{\sum_{\ell=1}^8 \exp(-\|x_i - p_\ell\|^2 / (2\sigma_\ell^2))},$$

where σ_j is the mean distance from control j to its three nearest controls. The six nearest same-group controls form the ARAP graph. Thus a player’s splats cannot be moved by any other player or by background controls.

Why global-plus-local motion and rigid MLS. Optimizing every control independently is a common deformation-graph choice, but fast bulk player motion made weakly observed controls lag and random-walk. In our comparison it reached 0.60 IoU at frame 10, versus 0.71 with a shared per-person translation plus local residual offsets. The shared term handles translation and the centroid-ray anchor re-acquires a player; the residual controls then express articulated motion. Rigid MLS and ARAP retain local shape better than independently moving splats, while still allowing a smooth non-rigid field.

At frame f , warm-start control offsets from the previous control velocity. Optimize a shared translation T_k per movable group and local control offsets δ_j for 50 Adam iterations, using four cameras/iteration and learning rates 0.020 m and 0.003 m respectively. Let $t_j = T_{g(j)} + \delta_j$, $p'_j = p_j + t_j$, and retain the fixed weights above. For Gaussian i ,

$$\bar{p}_i = \sum_j w_{ij} p_j, \quad \bar{q}_i = \sum_j w_{ij} p'_j, \quad M_i = \sum_j w_{ij} (p_j - \bar{p}_i)(p'_j - \bar{q}_i)^T.$$

With $M_i = USV^T$, set $R_i = VU^T$, flipping the last column of V if $\det(VU^T) < 0$. The baked position is $x'_i = R_i(x_i - \bar{p}_i) + \bar{q}_i$; orientation and degree-1 SH are also rotated by R_i . The fused CUDA operation evaluates this transform and its backward pass. The reported run does not blend separately optimized control rotations.

Every fifth frame, render per-instance weights, prompt SAM2 with each rendered box and centroid, and resolve overlap by highest rendered instance weight. Triangulate each mask centroid by area-square-root-weighted least-squares ray intersection. Use the resulting displacement to initialize T_k , and apply a Huber anchor of scale 0.3 m and weight 0.1 when enough views observe the identity. On intervening frames, no mask or anchor is fabricated. The deformation objective is

$$\mathcal{L} = \mathcal{L}_{\text{image}} + 0.3\mathcal{L}_{\text{BCE}}^{\text{instance}} + \mathcal{L}_{\text{ARAP}} + 0.01\mathcal{L}_{\text{accel}} + \mathcal{L}_{\text{anchor}},$$

where ARAP averages $\|(p'_a - p'_b) - R_a(p_a - p_b)\|^2 / \|p_a - p_b\|^2$ over graph edges and acceleration is normalized second-order control displacement. During deformation, background controls are frozen and the dynamic raster is depth-composited over a cached background.

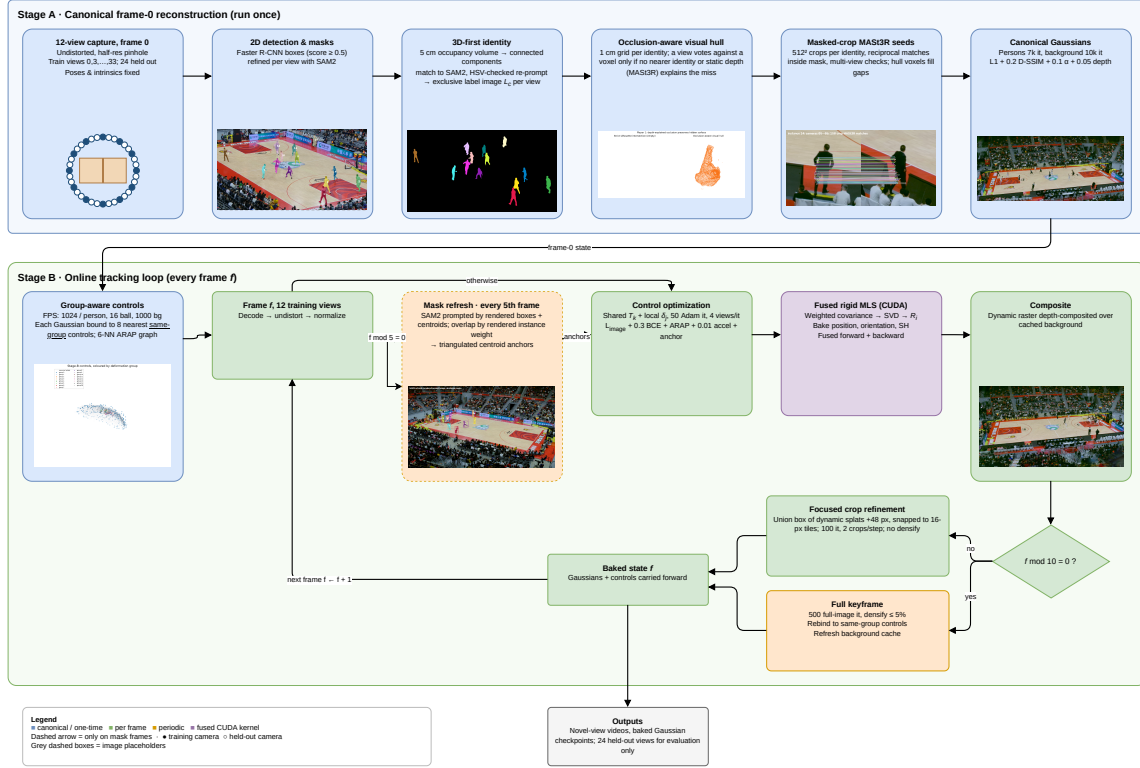


Figure 1: CourtFlow-GS pipeline. The editable diagram source is distributed with the report as courtflow_gs_pipeline.drawio.

2.3 Focused refinement

The MLS step supplies temporally coherent geometry but intentionally does not optimize Gaussian appearance or add new detail. A full-image 3DGS update after every frame is the usual alternative, but it spends most rasterization/backward work on the static court even though players cover less than 1% of an image. Focused refinement concentrates the same photometric capacity on the moving content, while periodic full-image keyframes correct accumulated drift and permit limited growth. This division is why the method can improve player detail without constantly destabilizing the control bindings or paying keyframe cost every frame.

Bake Gaussian and control state forward after every deformation. At a non-keyframe, project all dynamic Gaussians into every training camera, form their union box with 48 pixels of padding, snap it outward to the 16-pixel gsplat tile grid, shift the principal point by the crop origin, and run 100 refinement iterations on two crops/step. Tile alignment makes crop rasterization pixel-identical to the corresponding full render. This phase has no densification or pruning, so control bindings remain stable. Every tenth frame instead runs 500 full-image iterations, permits densification from iteration 50 every 50 steps until 60% of the schedule, caps growth at 5% per keyframe, rebinds each Gaussian to eight nearest same-group controls, and refreshes the background cache. Held-out cameras are never used in a loss.

2.4 Pseudocode

Algorithm 1 CourtFlow-GS online reconstruction

Require: Frame-0 images, synchronized training videos, fixed calibration

```

1:  $(state, controls) \leftarrow \text{BUILD CANONICAL SCENE}(frame_0, calibration)$ 
2: for each  $f$  in the video sequence do
3:    $images \leftarrow \text{DECODE UNDISTORT NORMALIZE}(trainingVideos, f)$ 
4:   if  $f \bmod 5 = 0$  then
5:      $masks \leftarrow \text{SAM2 REPROMPT}(images, state.dynamic)$ 
6:      $anchors \leftarrow \text{TRIANGULATE MASK CENTROIDS}(masks)$ 
7:   else
8:      $masks, anchors \leftarrow \emptyset, \emptyset$ 
9:   end if
10:   $offsets \leftarrow \text{OPTIMIZE CONTROLS}(L_1 + D\text{-SSIM}, masks, anchors, ARAP)$ 
11:   $state \leftarrow \text{BAKE FUSED RIGID MLS}(state, controls, offsets)$ 
12:  if  $f \bmod 10 \neq 0$  then
13:     $state \leftarrow \text{REFINE DYNAMIC CROPS}(state, images, 100)$ 
14:  else
15:     $state \leftarrow \text{REFINE FULL IMAGES}(state, images, 500)$ 
16:     $controls \leftarrow \text{REBIND}(state, controls); \text{CACHE BACKGROUND}(state)$ 
17:  end if
18:   $\text{SAVE DIAGNOSTICS}(state)$ 
19: end for

```

3 Results

Table 1: Quality summary. Temporal held-out samples coincide with keyframes.

Method	Full PSNR	Person PSNR	Person IoU	Measurement
Global 3DGS baseline	17.50	–	–	frame-0 held-out
Stage A canonical	19.32	18.34	0.788	frame-0 held-out
Stage B v1	18.29	15.00	0.70→0.30	keyframe held-out mean
Stage B v2 (old)	18.87	19.62	0.71→0.39	keyframe held-out mean
Focused refinement, frames 1–20	22.42	26.76	0.746	training-view mean
Final 300-frame diagnostic	20.07	22.44	–	held-out mean



Ground truth, view 13, frame 200



Earlier Stage B v2



CourtFlow-GS diagnostic

Figure 2: Same-camera comparison at held-out view 13, frame 200. Replace or add comparison rows as desired.

4 RTX 3080 runtime

Table 2: Runtime.

Component	RTX 3080 12 GB	RTX PRO 6000
MLS deformation/control optimization	1.95 s/frame	[measure on RTX PRO 6000]
Focused crop refinement	3.41 s/non-keyframe	[measure on RTX PRO 6000]
500-step full keyframe	23.16 s/keyframe	[measure on RTX PRO 6000]
Mask refresh, amortized	0.33 s/frame	[measure on RTX PRO 6000]
End-to-end diagnostic	7.61 s/frame (0.131 FPS)	[measure on RTX PRO 6000]
GPU SH colour + baked raster, 937×527	4.36 ms (229 FPS)	[measure on RTX PRO 6000]

The fused MLS/SVD path is a small fraction of tracking time; rasterization forward/backward dominates optimization. Baked-state GPU rendering is fast, while Python-to-ffmpeg transfer limits the current encoded-video exporter.

5 Limitations and next steps

Crowded late-game plays cause independent per-camera SAM2 identities to swap or disappear. In the final diagnostic, training IoU falls from 0.789 near the start to 0.001 at frame 299; held-out person PSNR falls from 23.86 to 20.59 dB. The highest-priority improvement is detector-led, multi-view identity association at keyframes. Additional work includes checkpoint consolidation or LOD for the 2.01M-Gaussian final state, unbiased inter-keyframe held-out evaluation, and direct GPU-to-NVENC output.